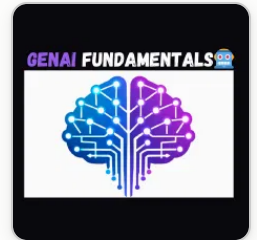


Practical GenAI engineering learning path ▾



I'm learning GenAI and want to understand these topics deeply enough for real-world engineering work.

Use the PDF as the main source for my course notes, but add your own explanations, examples, practical knowledge, and missing concepts where needed.

Teach me step by step, covering:

- What I need to learn
- How deeply I need to learn each topic
- Why it matters
- How it works
- Practical implementation
- What is enough for a GenAI engineer in 2026
- What can be skipped or learned later

Keep the learning path structured, practical, and beginner-friendly, without unnecessary jargon.

Architected layered learning path synthesizing fundamentals with practical engineering skills

Great primer to build from — it covers the core idea (tokenization → next-token prediction → autoregressive loop) but stays at the "no math" surface level, which is exactly right for step one. Let me expand this into a full engineering-grade learning path: what's in the slides, what's missing, how deep to go, and what actually matters for building GenAI systems in 2026.

I'll turn this into a structured reference doc you can keep coming back to.

Ran a command, created a file, read a file

This builds directly on your slides (tokens → next-token prediction → autoregressive loop) and fills in what a real engineering role needs on top: sampling params, attention/context window intuition, embeddings, RAG, tool calling/agents, and evals — each tagged Usage Standing ✓/26%
 ○ for how deep to go, plus an 8-week roadmap. RAG and evals are the two biggest gaps versus the deck, since they're what most real GenAI work is actually built around in 2026.



Genai engineer learning path

Document · MD

Download

teach me in sequence

Architected sequential teaching strategy with foundational concepts first

Let's do this properly — one concept at a time, building on the last. I'll teach it, give you something concrete to try or think about, then move to the next piece once it clicks.

Lesson 1: Tokens — What the Model Actually "Sees"

Forget words and sentences. The model never sees English. It sees a list of numbers.

The pipeline:

```
"ChatGPT is amazing!"  
  ↓ tokenizer (BPE algorithm)  
["Chat", "GPT", " is", " amazing", "!"]  
  ↓ each token maps to an ID  
[16932, 38, 318, 5678, 0]
```

That numeric list is the *only* thing the model works with internally. "Chat" isn't a word to it — it's ID 16932, a single point in a giant lookup table.

Three things that trip people up, and why they matter:

1. **Tokens ≠ words.** A rare word like "antidisestablishmentarianism" might split into 6+ small pieces. A common word like "the" is always one token. This is why token counts are unpredictable if you just estimate from word count.
2. **Tokenization is fixed before the model is trained.** It's a separate, earlier step — a fixed vocabulary (usually 100K–200K possible tokens) gets built once from a huge text corpus, and every model built on that tokenizer reuses it. The model itself never "chooses" how to split text.
3. **This is where cost and limits come from.** When someone says "this model has a 200K context window," they mean 200K *tokens*, not words or characters. When you're billed per token, this is the literal unit of billing.

● Usage Standing ✓ | 26%

Try this to make it concrete: think about how you'd expect these to tokenize, then I'll confirm:

- "running" — probably 1 token (common word)
- "unbelievably" — probably splits into pieces like ["un", "believ", "ably"]
- "claude-sonnet-5" — probably splits a lot, because hyphenated/technical strings aren't common in training text

Take a guess at how many tokens you think "The quick brown fox jumps over the lazy dog." would be (rough count), and tell me your reasoning — then we'll move to Lesson 2: what the model actually *does* with these token IDs (next-token prediction, for real this time, past the slide's surface version).

This study plan tracks the evolution of NLP from **counting words** to **mapping meaning** through context-aware models. It is designed to take you fr...

PASTED

I'm currently learning Generative AI and have started studying "Attention Is All You Need" and the Transformer architecture and HOW LLM works. Before diving into Transformers, I came across several important NLP foundations and historical concepts, such as:

- Tokenization
- One-Hot Encoding (OHE)
-
- Bag of Words (BoW)
- TF-IDF
- Embeddings and vectors
- Vector spaces
- Weights and representations
- Pretraining / base training
- Specialization
- Fine-tuning
- PEFT and related techniques

As a Computer Science student, I want to understand how these concepts connect and why the field evolved from one approach to another.

Teach me the complete progression from traditional NLP → embeddings → neural networks → attention → Transformers → modern LLM training and fine-tuning.

Focus on what, why, and how, including the intuition,  Usage Standing  26%  1 practical relevance. I want enough depth to genuinely understand Transformers and modern GenAI—not just memorize terminology.

help me design a custom study plan with lessons suitable for me based on the research below also you can add anything if you think might be better

Show more

Synthesized curriculum framework merging foundations through modern architectures

Good instinct pulling in that outline — it's a solid map of the field. Let me merge it with where we already started (tokens) and tighten it into a sequence built for CS-level understanding, not memorization. Then we'll keep going exactly where we left off.

Your Custom Study Plan

#	Topic	Core Question	Depth
0	Tokenization (started)	How does text become numbers at all?	
1	OHE → BoW → TF-IDF	How did we represent words <i>before</i> neural nets, and why did it fail?	
2	Word2Vec / GloVe (static embeddings)	How do we get vectors where meaning = geometry?	
3	RNNs & the bottleneck problem	Why did sequence order need a new kind of model — and where did it break?	
4	Attention (Q, K, V)	How does a model decide what to "look at" dynamically?	
5	The Transformer	How does attention become a full trainable architecture (self-attention, multi-head, positional encoding)?	
6	Pretraining objectives: GPT vs BERT	Why decoder-only "predict next word" won for the LLM era?	
7	Fine-tuning & specialization	How does a base model become an assistant? (instruction tuning, RLHF)	
8	PEFT / LoRA	How do we customize a 70B-parameter model without retraining it?	
9	Tie-back: modern engineering	How does all this map to prompting/RAG/agents (from your earlier doc)?	

Usage Standing

✓

 | 26%

Why this order: each stage exists **because the previous one failed in a specific, nameable way**. That failure→fix chain is the actual story of NLP, and it's the thing that makes all this stick instead of being a list of buzzwords. We'll hit that failure explicitly at every step.

One addition to your source list: I added **step 3 (RNNs)** even though it's not in your original doc's phase list — you can't really feel *why* attention was such a breakthrough unless you first see the problem it solved. I'll keep it light (🟡, no deep math), just enough to feel the "aha."

Quick close on where we left off: "The quick brown fox jumps over the lazy dog." tokenizes to about **10 tokens** — each word is one token here (all common English words), plus the period. Nothing splits into sub-pieces because every word is frequent in training data. Good instinct if you guessed close to word count — that's the normal case; sub-word splitting only shows up with rare/technical/foreign words.

Lesson 1: Before Meaning — OHE, Bag of Words, TF-IDF

Tokenization gives you a list of tokens. The next question computers had to answer: **how do you turn a token into a number a machine learning algorithm can actually compute with?** This is 1990s–2000s NLP, and it's worth understanding because the failures here directly motivated everything that came after.

One-Hot Encoding (OHE)

What: Build a vocabulary of every unique word (say, 50,000 words). Each word becomes a vector of length 50,000 — all zeros, except a single 1 at that word's position.

```
vocab = ["cat", "dog", "airplane", ...]    (50,000 words)
```

```
"cat"      → [1, 0, 0, 0, ...]
```

```
"dog"      → [0, 1, 0, 0, ...]
```

```
"airplane" → [0, 0, 1, 0, ...]
```

The CS insight that matters: this is a *sparse* representation — one non-zero value in tens of thousands of slots. And critically: **the distance between any two words is identical**. Measure the distance between "cat" and "dog," then between "cat" and "airplane" — mathematically, they're exactly the same distance apart. The vector has zero notion that cats and dogs are more alike than cats and airplanes. Every word is equally foreign to every other word. That's the failure this whole field spent 20 years fixing.

Bag of Words (BoW)

What: To represent a whole *document* (not just one word), sum up the one-hot vectors of every word in it — you get a histogram of word counts, ignoring order entirely.

● Usage Standing ✓ | 26%

"the cat sat" and "sat the cat" → produce the identical vector

Why this matters (and why it's a real failure, not just a simplification): "not good" and "good" can end up looking nearly identical to a BoW model, because word order and negation are thrown away completely. This is a genuine, practical bug — early sentiment classifiers built on BoW would misclassify negated sentences constantly.

Where it's still genuinely useful today: BoW-style representations are still fine for quick, cheap document classification/spam filtering where you don't need nuance — it's fast and interpretable. 🟡 -level knowledge — know it exists, know its ceiling, don't over-invest.

TF-IDF (Term Frequency-Inverse Document Frequency)

What: An upgrade to BoW that weights words by how *informative* they are, not just how often they appear.

- **TF (term frequency):** how often a word appears in *this* document.
- **IDF (inverse document frequency):** a penalty for words that appear in *most* documents (like "the," "is," "and") — the more common a word is across your whole corpus, the lower its weight.

$$\text{TF-IDF}(\text{word}) = \text{TF}(\text{word in this doc}) \times \text{IDF}(\text{word across all docs})$$

The intuition, concretely: in a set of product reviews, the word "the" appears in nearly every review — TF-IDF crushes its weight toward zero. The word "defective" appears in only a few — TF-IDF boosts it, because its presence is *informative*. This is the direct ancestor of things like search relevance ranking (it's part of how early search engines scored keyword matches).

Why this still matters practically in 2026: TF-IDF is still a legitimate, fast baseline for keyword-based search and is often combined with modern vector search in "hybrid search" systems (we'll hit this again later, in RAG). Don't dismiss it as purely historical — it's 🟡, not ○.

The common thread — why all three eventually get replaced

OHE, BoW, and TF-IDF share one core limitation: **they represent words by counting or flagging them, never by understanding them.** There is no mechanism anywhere in this pipeline that could ever learn "cat" and "dog" are semantically related — the representation is purely statistical bookkeeping, not meaning.

That exact gap — *representations with no notion of meaning* — is the problem Word2Vec was built to solve, which is Lesson 2.

Usage Standing ✓ | 26%

Quick check before we move on (this is the kind of question that tells me if the intuition landed): using TF-IDF logic, would the word "**diagnosis**" get a *higher* or *lower* weight in a single medical report, compared to the word "**patient**," assuming both appear the same number of times in that report but "patient" appears in almost every report in the hospital's archive and "diagnosis" appears in far fewer? Take a guess and tell me your reasoning — then we'll move to **Lesson 2: Word2Vec and the birth of dense embeddings**, where meaning finally enters the picture.

Diagnosis get higher weight. Because as single document might be appear same number of time but in a large corpus diagnosis appears less in report .

Exactly right, and your reasoning is precise. Let's make the mechanism explicit so it's locked in:

- **TF** is equal for both words in this document (same count) — so TF alone can't distinguish them.
- **IDF** is where the split happens: "patient" shows up in almost every report in the archive → its IDF is *low* (common word, not very informative). "diagnosis" shows up in fewer reports → its IDF is *higher* (rarer across the corpus, so more informative when it does appear).
- **TF-IDF = TF × IDF** → same TF, but "diagnosis" gets multiplied by a bigger IDF → **higher final score**.

That's the whole mechanism. You've got it.

Lesson 2: Word2Vec — Where Meaning Enters the Picture

Everything in Lesson 1 (OHE, BoW, TF-IDF) represented words by **counting**, never by **understanding**. Word2Vec (2013, Google) was the first mainstream approach where a model actually *learns* what a word means, from data, with no human labeling.

The core idea: "you shall know a word by the company it keeps"

This is a linguistics idea from the 1950s (J.R. Firth) that Word2Vec turned into an algorithm. The hypothesis: **words that tend to appear in similar contexts probably have similar meanings**.

Think about it yourself: "dog" and "cat" both frequently appear near words like "pet," "feed," "vet," "walk." "airplane" almost never appears near those words — it shows up near "flight," "runway," "pilot." Nobody labeled these words as similar. The *pattern of their neighbors* is the signal.

Usage Standing, ✓ | 26%

How Word2Vec actually trains (two ways to frame the same idea)

Word2Vec is a small neural network trained on one made-up task, not because the task itself is useful, but because solving it *forces* the network to learn something useful about meaning as a side effect.

Skip-gram: given one word, predict its surrounding neighbor words.

Input: "fox"

Task: predict nearby words → "quick", "brown", "jumps", "over"

CBOW (Continuous Bag of Words): the reverse — given the surrounding words, predict the missing middle word.

Input: "quick", "brown", ___, "jumps", "over"

Task: predict the missing word → "fox"

Neither of these tasks is something anyone actually needs solved. But here's the trick, and it's the important CS concept: **to get good at this prediction task, the network is forced to compress each word into a dense vector (say, 300 numbers) that captures its typical usage context.** The predicting task is just the training pressure — the useful byproduct is the vector.

This is the same underlying philosophy as your slides' "next-token prediction" — a simple, repeatable prediction task, done at massive scale, produces something far more useful than the task itself.

Dense vs. sparse — the actual upgrade

This is the direct fix to the OHE failure from Lesson 1:

OHE "cat": [0,0,0,1,0,0,0,0,0,0,0, ... 49,990 more zeros ...] ← 50,000
dims, sparse

Word2Vec "cat": [0.21, -0.55, 0.87, 0.03, -0.12, ...] ← 300 dims,
dense

Every number in the Word2Vec vector carries information (no wasted zeros), and — this is the key property — **distance in this space now means something.** `distance(cat, dog)` is small. `distance(cat, airplane)` is large. This is the property OHE could never have, by construction.

Vector arithmetic — the famous demonstration

● Usage Standing ✓ | 26%

king - man + woman $\approx$ queen

Why this isn't a party trick: it shows the model learned *directions* in the vector space that correspond to real semantic relationships (in this case, something like a "gender" direction, and separately a "royalty" direction). This was the proof, in 2013, that these vectors weren't just clustering similar words — they'd learned structured relationships. This result is what convinced the field that dense embeddings were the future.

The limitation — and it's a big one

Word2Vec produces **one fixed vector per word**, decided once during training and never changed again.

"I deposited money at the bank."	→ "bank" gets vector X
"I sat by the river bank."	→ "bank" gets the exact same vector X

The model has no way to know which "bank" you mean — it committed to one vector per word regardless of context. This is called a **static embedding**, and it's the single most important limitation to understand, because **it is precisely the gap Attention and Transformers were built to close**. Hold onto this — it's the thread connecting Lesson 2 to Lesson 4.

(GloVe, mentioned in your source doc, is worth knowing exists —  — it's a contemporary of Word2Vec that learns embeddings from global word co-occurrence statistics across the whole corpus rather than local context windows. Same static-embedding limitation applies. Not worth deep separate study; the intuition above covers it.)

Quick check: given what you now know about *why* Word2Vec vectors are static, can you predict what property a *good* solution would need — i.e., what would "bank" need to be able to do differently, if the model is going to correctly distinguish "river bank" from "money bank"?

Take a guess, then we'll move to **Lesson 3: RNNs and the bottleneck problem** — the (short) detour that sets up why Attention was such a breakthrough in Lesson 4.

a good solution must understand the context and remember what previous words/sentences. For example He is sitting at a river bank and he went bank to withdraw money. Both are not same

Usage Standing  26%

Right idea, and your example nails the ambiguity. Let's sharpen it into the precise property, because this exact phrasing is what will make Attention click in Lesson 4:

It's not just "remembering previous words" (that's closer to what RNNs try to do, which we're about to cover) — the specific fix is: **the vector for "bank" needs to be computed fresh, on the fly, using its neighboring words each time, instead of being looked up once and reused forever.**

- Word2Vec: "bank" → always vector **X**, no matter the sentence (a fixed dictionary lookup).
- What you actually need: "bank" in "sitting at a river bank" → vector **X**. "bank" in "went to the bank to withdraw money" → a *different* vector **Y**, generated dynamically because "river" and "withdraw money" are different neighbors pulling the meaning in different directions.

That's the definition of a **contextual embedding** — the same word, different vector, depending on context. You've essentially derived, from first principles, the entire motivation for Transformers before we've even covered them. That's exactly the intuition to hold onto.

Lesson 3: RNNs and the Bottleneck Problem (short detour)

Before Attention existed, the field's attempt at "context-aware" processing was the **Recurrent Neural Network (RNN)**. It's worth 10 minutes of understanding — not building — because the *specific way it failed* is what makes Attention's design feel inevitable rather than arbitrary.

The idea

An RNN reads a sentence one word at a time, left to right, and maintains a single running "memory" vector (called the **hidden state**) that gets updated at every step.

```
"The" → update memory → h1
"cat" → update memory (using h1) → h2
"sat" → update memory (using h2) → h3
...
```

Conceptually: it's like reading a sentence while continuously summarizing everything you've read so far into one notebook page — and every new word **forces you to cram it into that same single page, overwriting some of what was there.**

Usage Standing ✓ | 26%

The bottleneck

By the time the RNN reaches word 50 in a long sentence, the hidden state is supposed to still contain useful information about word 2. But it's just one fixed-size vector, repeatedly overwritten — information from early words gets diluted or lost, the same way old details fade the more you keep summarizing.

Concretely: for a sentence like *"The trophy doesn't fit in the suitcase because it is too big"* — resolving what "it" refers to requires clearly remembering "trophy" from way earlier. RNNs, especially on longer sequences, systematically struggle with exactly this kind of long-range dependency, because everything has to survive being repeatedly compressed into that single evolving vector.

There's a second, very practical CS problem: RNNs process **strictly sequentially** — you can't compute word 10's update until you've computed word 9's. This means **no parallelization**, which made RNNs slow to train at the scale modern LLMs eventually needed. Keep this in mind — it resurfaces in Lesson 5 as one of the Transformer's biggest wins (it processes tokens in parallel).

The reframe that sets up Attention

The fix the field landed on: **stop forcing everything through one fixed-size memory bottleneck. Instead, let the model look directly back at every previous word individually, and decide — for each new word — which of those earlier words actually matter right now.**

That's it. That single sentence *is* the concept of Attention. You're now ready for Lesson 4, where we make it mechanical: Queries, Keys, and Values — the actual computation behind "deciding which earlier words matter right now."

Quick check before we go there: in the sentence *"The trophy doesn't fit in the suitcase because it is too big,"* if a model were using attention instead of an RNN's single memory vector, which word(s) do you think "it" would need to attend to strongly, and — just intuitively, no math yet — why would attention have an easier time with this than the RNN's bottleneck approach?

trophy

Right — "it" needs to attend strongly to "trophy" (not "suitcase," even though both are nearby nouns — the correct resolution here actually depends on world knowledge about size and fitting, but structurally, attention gives the model the *ability* to link "it" directly back to "trophy" regardless of distance).

Usage Standing ✓ 26%

Why attention has an easier time with this than the RNN: the RNN can only pass "trophy" forward by keeping it alive inside one continuously-overwritten memory vector across every intermediate word ("doesn't," "fit," "in," "the," "suitcase," "because"). Attention doesn't rely on that relay at all — when the model processes "it," it can reach back and connect **directly** to "trophy," as if every earlier word is sitting in front of it at once, no matter how far back it is. Distance stops being a cost. That's the entire structural advantage.

Lesson 4: Attention — Queries, Keys, and Values

This is the mechanism that makes "look back and decide what matters" computable. It's the most important lesson in this whole sequence — everything downstream (Transformers, GPT, modern LLMs) is this operation, repeated and scaled up.

The librarian analogy first, math second

Imagine you're the word "it", and you need to figure out what you refer to. You walk into a room containing every other word in the sentence. Here's the roleplay:

- **Query (Q):** *your own question* — "I'm the word 'it'; what noun do I refer to?" This is a vector representing what you're looking for.
- **Key (K):** *every other word's label* — each word in the sentence holds up a sign describing what kind of information it offers. "trophy" holds up a key saying, roughly, "I'm a physical object, singular, thing that can be large/small."
- **Value (V):** *the actual content* each word carries — the substance you'd retrieve if you decide that word is relevant.

The mechanism, step by step:

1. Compare your Query against every other word's Key (a similarity/match score — computed via dot product, but the math isn't the point right now).
2. Words whose Key strongly matches your Query get a **high score**; irrelevant words get a **low score**.
3. Turn those scores into weights that sum to 1 (this step is called **softmax** — same "turn scores into a probability distribution" idea as your very first slide showing Paris → 0.85, London → 0.05).
4. Take a **weighted sum of every word's Value**, using those weights.

```
"it" attending to the sentence:
trophy    → key matches well    → weight 0.75
suitcase  → key matches some    → weight 0.15
fit       → key barely matches → weight 0.05
...rest   → ~0.05 combined
```

New vector for "it" = $0.75 \times (\text{trophy's value}) + 0.15 \times (\text{suitcase's value}) + \dots$

The output is a **brand new vector for "it"** — no longer just "the generic word 'it'," but "it, *informed mostly by trophy*." This is a **contextual embedding**, computed fresh for this exact sentence. This is precisely the property you derived yourself two lessons ago, now made mechanical.

Why three separate things (Q, K, V) instead of one?

This trips people up, so here's the CS reasoning: each word plays **three different roles simultaneously** in this computation — it asks a question (as Query), it's evaluated by others (as Key), and it offers information if selected (as Value). These are learned as three separate transformations of the same input vector, because "what I'm looking for," "how I advertise myself to others," and "what I actually contain" are genuinely different jobs, and forcing one vector to do all three would be a bottleneck all over again — the exact failure mode from Lesson 3.

Self-attention, specifically

What we just walked through — a sentence attending to *itself* (every word comparing against every other word in the same sentence, including itself) — is called **self-attention**, and it's the specific variant used in Transformers/GPT. Every word does this Q-K-V process in parallel, simultaneously, for every other word. That's the parallelization win over RNNs mentioned in Lesson 3 — none of these computations depend on finishing a previous word first.

Quick check (this is the one that confirms Q/K/V actually landed, not just the analogy): in the sentence "**The cat chased the mouse because it was hungry**," when the model computes attention for the word "**it**," which word's **Value** vector do you expect to dominate the weighted sum — and separately, what is doing the matching to find that word (Query of what, against Key of what)?

Usage Standing ✓ | 26%

The cat will dominate. First each word will be comparing each others including itself. SO there is a key for each word. It will find the related words and after that

determine the weighted sum .All those works done in parallelly query , finding Key and values

Good — you've got the mechanism and the parallelism right. Let me sharpen the roles just so they're crisp before we move on, since "which vector is doing which job" is the part people blur:

- **"it" produces the Query** — it's the one asking the question ("what do I refer to?").
- **Every word, including "cat," "mouse," "chased," etc., produces a Key** — each one is offering its own label to be compared against.
- The Query from "it" gets compared against **every Key**, and "cat"'s Key scores highest (semantically, "cat" is the kind of thing that can be "hungry" — a mouse being chased is less plausibly the hungry one, that's the world-knowledge signal the model has picked up from training data).
- Because "cat"'s Key wins the comparison, "cat"'s **Value** gets the largest weight in the final weighted sum → the new vector for "it" ends up dominated by "cat"'s content.

So: **Query = asking, Key = matching, Value = what actually gets pulled in.** That's exactly right in your answer — just naming which word plays which role at each step. You're solid on this.

Lesson 5: The Transformer — Turning Attention Into a Full Architecture

Attention alone (Lesson 4) is one operation. The Transformer (2017, "*Attention Is All You Need*") is the full architecture that wraps that operation into something trainable at scale. Three additions turn "attention" into "Transformer":

1. Multi-Head Attention — running attention several times in parallel, differently

A single attention pass forces one Query/Key/Value comparison per word. But language has multiple *kinds* of relationships happening at once — grammatical (subject-verb agreement), semantic (what does "it" mean), positional (nearby words often relate), and more.

The fix: instead of one attention computation, the Transformer runs several (e.g., 8, 16, or more) **in parallel**, each with its own separately-learned Q/K/V transformations. Each one is called a **head**.

Head 1 might learn to focus on: grammatical structure (subject ↔ verb)
 Head 2 might learn to focus on: pronoun ↔ noun resolution ("It" ↔ trophy) 26%
 Head 3 might learn to focus on: nearby-word relationships
 ...

Nobody *tells* each head what to specialize in — this division of labor emerges naturally from training, because each head has independent parameters and gets rewarded for contributing something useful to the final prediction. After all heads run, their outputs get combined into one vector. This is why it's called **multi-head attention** — many "perspectives" on the same sentence, computed simultaneously, then merged.

Why this matters practically: this is *the* reason Transformers generalize so well across grammar, meaning, coreference, and world knowledge simultaneously — no single mechanism has to do all of that alone.

2. Positional Encoding — telling the model about word order

Here's a subtle but important problem: self-attention, as described in Lesson 4, treats the sentence as a **set**, not a sequence. Nothing about "compare every word's Query against every other word's Key" inherently knows that "cat" came before "chased." If you scrambled the word order, the raw attention computation wouldn't notice.

But word order obviously matters — "*dog bites man*" and "*man bites dog*" must not look the same to the model.

The fix: before the sentence enters the attention layers, the model adds a **positional encoding** vector to each word's embedding — a mathematical signal (commonly built from sine and cosine waves at different frequencies) that's unique to each position in the sequence (position 1, position 2, position 3...). This gets added directly onto the word's vector, so "order" becomes baked into the numbers themselves rather than being tracked separately.

You don't need to derive the sine/cosine formula — the concept to hold onto is: **position becomes part of the input data, not a separate mechanism**, which keeps the parallelization benefit from Lesson 3 intact (you're not processing sequentially to preserve order — you're encoding order directly into the numbers).

3. Stacking layers — depth

One layer of multi-head attention produces one round of "contextualized" vectors. Real Transformers stack many such layers (e.g., dozens), each one taking the previous layer's output and refining it further. Early layers tend to capture more local/grammatical patterns; deeper layers capture more abstract, longer-range meaning. This is the same "depth = more abstraction" principle you may have seen in image-recognition neural networks — early layers catch edges, later layers catch faces. Same idea, applied to language.

Encoder vs. Decoder — the fork that matters most for you

● Usage Standing ✓ | 26%

Your source doc's Phase 5 flagged this, and it's genuinely one of the most important distinctions for understanding modern LLMs:

- **Encoder** (used in BERT): looks at the **entire sentence at once, in both directions** — it can see words before *and* after the current word. Good for *understanding* text (classification, embeddings), bad for *generating* text (it would be cheating to see the future word you're supposed to predict).
- **Decoder** (used in GPT and virtually every modern chat-style LLM, including Claude): can only look **backward** — at the current word and everything before it, never ahead. This is called **causal** or **autoregressive** attention, and it's *exactly* the mechanism behind your very first slide: predict the next token using only what's come so far.

Encoder (BERT): "The [MASK] sat on the mat" → can see both "The" and "sat on the mat" to guess the masked word

Decoder (GPT): "The cat sat on the ___" → can only see leftward context, must predict forward

Modern general-purpose LLMs (GPT, Claude, Llama, etc.) are **decoder-only**. This is the direct architectural reason your slide deck's "next-token prediction" framing is accurate — it's not a simplification, it's literally what a decoder-only Transformer does at inference time, one attention-computation-per-token, repeated in a loop.

Quick check before Lesson 6 (pretraining: GPT vs. BERT objectives, in more depth): given that a decoder can *only* look backward, why do you think this restriction is *necessary* for a model whose job is to generate open-ended text — what would go wrong if a decoder-style model were allowed to peek ahead during training?

Decoder will predict forward but can only see left because it needs the context of the full data and based on this it has to answer. Cause it will helps to understand query and genrates values by calculating keys

Untangled causal masking mechanics and training efficiency nuances

Let's untangle this — you're right that it's connected to Q/K/V, but the actual reason is more specific and important, so let's nail it precisely.

The real problem: training and inference have to match.

At **inference time** (when you're actually using the model), a decoder generating text genuinely does *not* know the future — it's writing one token at a time, and token 6 hasn't been decided when it's generating token 5. There's no "peek ahead," because the future literally doesn't exist yet.

Usage Standing ✓ | 26%

Now think about **training**. For efficiency, the model isn't trained one token at a time in a slow loop — the whole target sentence is fed in at once, and the model predicts *all* positions simultaneously (much faster to train this way). But here's the trap: if attention were allowed to look in both directions during this training pass, then when predicting word 5, the model could just **peek at word 5 directly sitting right there in the input** and copy it. It wouldn't need to learn anything — it would get 100% accuracy by cheating, and learn nothing useful, because at real inference time that shortcut won't exist.

The fix — causal masking: when computing attention for position 5, the model is only allowed to compute Query-Key matches against positions 1–5, never 6 onward. Positions ahead are literally masked out (their attention score is forced to zero, no matter how well the Key would've matched). This forces the model to genuinely predict from left-context only, during training, matching exactly what it'll have to do at inference.

So the mechanism (Q/K/V) is the same one from Lesson 4 — you were right to connect it there — the addition is just: **certain Key comparisons are blocked entirely, based on position**, so training can't cheat by seeing the answer.

Lesson 6: Pretraining — What GPT and BERT Are Actually Trained To Do

Now that you know encoder vs. decoder architecturally, here's what each one is actually *trained* on — the specific task, the loss, and why it matters for what each model ends up good at.

BERT — Masked Language Modeling (MLM)

The task: take a sentence, randomly hide (mask) ~15% of the words, and ask the model to guess them, using context from **both sides**.

Input: "The [MASK] sat on the mat and purred softly"
Target: "cat"

Because BERT is an encoder, it can legally see "sat on the mat and purred softly" (the right side) when guessing — that's allowed, since it's not generating text left-to-right, it's just filling in a blank once, given the whole sentence.

What this makes BERT good at: deep, bidirectional *understanding* of a fixed piece of text — classification, sentiment analysis, extracting entities, producing high-quality embeddings for semantic search. **What it's bad at:** actually generating new open-ended text, because it was never trained to produce a sequence one step at a time — it fills in blanks, it doesn't write paragraphs.

● Usage Standing ✓ | 26%

GPT — Causal Language Modeling (next-token prediction)

The task: exactly your slide from day one — given everything so far, predict the single next token, left to right, over and over.

Input: "The cat sat on the"
Target: "mat"

This is trained on effectively the entire internet's worth of text, with no manual labeling required — the "label" for any position is just *whatever the actual next word was in the real text*. This is why it's called **self-supervised learning**: the data labels itself.

What this makes GPT (and Claude, Llama, etc.) good at: open-ended generation — because generation *is* the training task, not a side effect. This is why essentially every modern chat-based LLM is decoder-only: the product (write text, hold a conversation, produce code) is a direct match for the training objective.

Why this distinction actually matters for your engineering work

- If you ever need to build **semantic search or classify/embed text**, encoder-style models (BERT-family, or dedicated embedding models trained similarly) are usually the right tool — smaller, faster, and specialized for exactly that.
- If you need to **generate** anything — chat, summarize, write code — decoder-only models (GPT-family, Claude) are the right tool.
- Some architectures (T5, original Transformer) use **both** encoder and decoder together (encoder reads the input fully, decoder generates the output) — mainly used for translation-style tasks where you fully read one sequence before producing a different one. Good to know it exists (🟡), not essential to go deeper.

The two-stage process: pretraining → the model that "just predicts text"

One more thing worth being precise about, because it sets up Lesson 7 perfectly: what we just described — training on next-token prediction over huge amounts of internet text — produces what's called a **base model** (or "pretrained model"). At this stage, the model is extremely good at *predicting plausible continuations of text*, but it is **not yet a helpful assistant**. Ask a raw base model a question, and it might just continue the pattern by generating more questions, or completing it like it saw in some forum post, rather than actually answering you helpfully.

Base model input: "What is the capital of France?"

Base model output: "What is the capital of Germany? What is the capital of Italy?"

(it's just continuing a pattern that looks like a quiz list – because that's a plausible continuation, and plausibility is all it was trained on)

Turning that raw prediction engine into something that reliably answers, follows instructions, and behaves like an assistant is a **separate stage** — that's fine-tuning, which is Lesson 7.

Quick check before we get there: given that a base model is just trained to produce "plausible continuations," why do you think showing it lots of examples of the format `[question] → [helpful, direct answer]` would push it toward becoming an assistant — what exactly is changing about what counts as "plausible" to it?

i could not get the questions and answer to thinl

No problem — let's slow this one down with a very concrete before/after, since it's really just extending an idea you already understand well.

Remember what "plausible continuation" means for the base model. It learned, from huge amounts of raw internet text, "given this text, what usually comes next?" If its training data included forum posts, quiz lists, unfinished articles, etc., then given `"What is the capital of France?"`, a *statistically plausible* next thing — based on everything it saw during training — might genuinely be another question, because that pattern (a list of quiz questions) appeared a lot in its training data. It's not being unhelpful on purpose. It has no concept of "answer the user helpfully" yet — it only has a concept of "continue the pattern the way similar text usually continued."

Now, fine-tuning changes what "the pattern" looks like. After pretraining, you take the same model and train it *further*, but this time only on a curated dataset that looks like this, thousands of times over:

● Usage Standing ✓ | 26%

[Question] → [Direct, helpful answer]

"What is the capital of France?" → "The capital of France is Paris."

"How do I reverse a list in Python?" → "You can use `list.reverse()` or `list[::-1]`..."

"Summarize this paragraph: ..." → "[a concise summary]"

The training mechanism is **identical** to pretraining — it's still just next-token prediction, still adjusting the model's internal weights so it gets better at predicting the "correct" next tokens. The only thing that changed is **what data it's being shown**. Because every single example now demonstrates "question → helpful direct answer," the model's *statistics* shift: "answer directly and helpfully" becomes the new dominant pattern, instead of "continue like a random forum post."

Put simply: the model didn't gain a new ability or new understanding of "helpfulness" as a concept. Its sense of *what a plausible continuation looks like* got reshaped, because you changed the examples it's learning that pattern from. Same mechanism (next-token prediction) — different training data — different resulting behavior.

Does that version land? If yes, tell me in your own words what fine-tuning is actually doing to the model — just a sentence or two — and then we'll move to **Lesson 8: PEFT and LoRA**, which answers the very practical question: given that fine-tuning a 70-billion-parameter model in the normal way would be absurdly expensive, how do people actually do this efficiently?

The fine tuning showing the questions and answers more directly .In training when it show this it learns from it

That's the right idea — let's tighten the wording just slightly so it's precise: fine-tuning keeps training the *same* model on a curated set of question→answer examples, and because next-token prediction is still the mechanism, the model's sense of "what's the plausible next token" shifts toward "give a direct, helpful answer" instead of "continue like random internet text." Good — you've got it.

Lesson 7 (brief bridge): Full Fine-Tuning — and Why It's a Problem at Scale

Usage Standing 26%

Before PEFT/LoRA make sense, you need to feel the actual pain point they solve.

What "full fine-tuning" means: taking every single one of a model's parameters — for a large model, that's tens or hundreds of **billions** of individual numbers — and updating all of them during this additional training stage.

Why this is expensive, concretely:

- You need to store the model's weights in GPU memory (billions of numbers).
- During training, you *also* need to store gradients and optimizer state for every one of those parameters — in practice this multiplies memory needs several times over. A model that takes ~140GB just to load can need 500GB+ to fully fine-tune.
- That means specialized, expensive, multi-GPU hardware — out of reach for most individual engineers or small teams.
- And if you want a *second* custom version of the model (say, one fine-tuned for legal text, another for customer support), you'd need to store **another full multi-hundred-gigabyte copy** of the entire model, since all its weights changed.

This is the exact "too slow and expensive" problem your source doc flagged. It's not a minor inconvenience — it's the reason most people and companies never do full fine-tuning at all.

Lesson 8: PEFT and LoRA — Fine-Tuning Without Retraining Everything

PEFT (Parameter-Efficient Fine-Tuning) is the umbrella idea: *instead of updating all the model's billions of parameters, freeze almost everything, and only train a small additional set of parameters.* **LoRA (Low-Rank Adaptation)** is the specific, most widely-used technique for doing this.

The intuition first

Here's the key insight LoRA is built on: when you fine-tune a model for a specific task, the *change* you actually need to make to its weights tends to be much simpler/smaller than the weights themselves — it doesn't need a full, complex rewrite, just a small, targeted "nudge" in a particular direction.

Think of it like this: imagine the model's full knowledge is a giant, extremely detailed painting. Full fine-tuning would mean repainting the entire canvas from scratch to adjust the style slightly. LoRA instead says: *keep the original painting completely untouched, and just lay a thin, semi-transparent overlay on top* — a much smaller, simpler layer that shifts the final appearance, without ever modifying the original canvas underneath.

How it works, mechanically

1. **Freeze the original weights entirely.** Every one of the billions of original parameters stays exactly as-is, untouched, for the whole process.

● Usage Standing ✓ | 26%

2. For specific weight matrices inside the model (typically the ones inside the attention layers — your Q, K, V transformations from Lesson 4, notably), **inject two small new matrices** alongside the original frozen one.
3. These two small matrices are deliberately **low-rank** — meaning they're much smaller/simpler than the original weight matrix. If the original matrix is, say, 4096×4096 (~16 million numbers), the two LoRA matrices might multiply out to something like 4096×8 and 8×4096 (~65,000 numbers combined) — a tiny fraction of the size.
4. **Only these small new matrices get trained** during fine-tuning. The original weights never move.
5. At inference time, the small matrices' output gets added on top of the original frozen weights' output — that's the "overlay" combining with the "original painting."

Original weight matrix W (frozen, untouched, huge)
 +
 Small LoRA matrices $A \times B$ (tiny, trainable)
 =
 Effective behavior of the fine-tuned model

Why this solves the actual problem from Lesson 7

- **Memory/compute:** you're now only computing gradients and storing optimizer state for a tiny fraction of parameters (often <1% of the total) — this is why LoRA fine-tuning can run on a single consumer or mid-tier GPU, instead of needing a data-center cluster.
- **Storage:** since the original model is untouched, you don't need to store a whole new multi-hundred-gigabyte copy per task. You just store the small LoRA matrices (often just a few megabytes) for each specialization. Want a legal-document version and a customer-support version? Keep one frozen base model, and swap in different tiny LoRA "overlays" depending on the task.
- **Quality tradeoff:** LoRA doesn't always match full fine-tuning's ceiling on very hard, large-scale adaptation tasks, but for the vast majority of practical customization needs (tone, format, domain vocabulary, following a specific style), it gets remarkably close, at a tiny fraction of the cost — which is exactly why it's the dominant approach used in practice today, not just a cheaper compromise people settle for.

Where this fits in your overall map now

You now have the full progression your original source doc asked for, Usage Standing  to 26% end:

OHE/BoW/TF-IDF (counting, no meaning)
 ↓ failure: no notion of similarity
Word2Vec/GloVe (dense, meaningful, but static)
 ↓ failure: one vector per word regardless of context
RNNs (sequential, context-aware, but bottlenecked)
 ↓ failure: long-range info gets lost, no parallelization
Attention / Q,K,V (dynamic, direct, parallel context)
 ↓ formalized into
Transformer (multi-head attention + positional encoding + depth)
 ↓ trained via
Pretraining (GPT: next-token prediction → base model)
 ↓ reshaped via
Fine-tuning (question→answer data → helpful assistant behavior)
 ↓ made efficient via
PEFT / LoRA (small trainable overlays, frozen base weights)

Quick check to close this out: in your own words, what is the *one-sentence* difference between what full fine-tuning changes versus what LoRA changes? Once you've got that, you're actually done with the full "traditional NLP → Transformers → modern LLM training" arc from your original study plan — the next natural step (if you want it) is connecting this to the *engineering* side: how you actually use pretrained/fine-tuned models in real products (prompting, RAG, tool use) — which is the roadmap from the very first document you uploaded. Want to keep going into that, or spend more time solidifying anything here first?

i am confused about after the pretraining how the answer is generated in claude
ior other llm

Synthesized explanation of LLM inference and generation mechanisms

Good question to pause on — this is where "training" and "actually answering you right now" get mixed up, and it's worth separating cleanly. Let's rebuild it from what you already know.

Two Completely Different Phases

Training (pretraining + fine-tuning, everything we just covered in Lessons 6–8) happens **once**, ahead of time, offline, on Anthropic's servers. It's where the model's weights — the

● Usage Standing ✓ | 26%

billions of numbers inside those attention layers — get set. Once training is done, the weights are **frozen**. They don't change while you're chatting with Claude.

Inference (what happens when you send a message) is a completely separate process that happens **every time you send a message**, using those already-frozen weights. This is generation, and it's exactly your very first slide, now with every mechanism underneath it filled in.

Here's the confusion, named directly: pretraining teaches the model *how to predict well in general*. It doesn't "store" your answer anywhere. When you ask a question, the model isn't looking up a saved response — it's running the exact same next-token-prediction computation, live, using the frozen weights as a very sophisticated calculator.

What Actually Happens When You Send a Message to Claude

1. Your message → tokenized → token IDs (Lesson 1)
2. Token IDs → passed through the Transformer (Lesson 5)
 - every layer runs self-attention (Q,K,V) (Lesson 4)
 - produces a probability distribution over every possible next token (your very first slide: Paris 0.85, London 0.05...)
3. One token is sampled from that distribution (temperature/top-p from your first document)
4. That new token gets appended to the input
5. The WHOLE sequence (original input + new token) is run through the model AGAIN, from scratch
6. Repeat steps 2-5, one token at a time, until the model produces a special "stop" token

That's it. That loop — steps 2 through 6 — is the entirety of "generating an answer." There is no separate "thinking module" that plans the whole answer first. Each token is a fresh forward pass through the same frozen network, conditioned on everything generated so far (including the tokens Claude itself just produced).

Why this matters — and answers something you might not have asked yet

This is *why* asking Claude to "think step by step" or "show your reasoning" actually improves answers on hard problems: the model is quite literally using its own previously-generated tokens as additional context for predicting what comes next. If it writes out reasoning steps, those steps become part of the input for predicting the final answer — genuinely useful extra context, not just a stylistic display. This is a direct, mechanical consequence of the loop above, not a separate feature bolted on.

The piece we haven't covered yet: how it got so good at being "helpful and safe" specifically

Lesson 7's fine-tuning (question→answer pairs) gets a model to answer directly instead of rambling. But real assistants like Claude go through one more training stage beyond that, worth naming even though it's 🟡-depth for you right now:

RLHF (Reinforcement Learning from Human Feedback) — after basic instruction fine-tuning, humans rank multiple model outputs for the same prompt (this response is better than that one). A separate model learns to predict those human preferences, and then the main model gets further adjusted to produce outputs more like the ones humans preferred — better tone, more honest, safer, more genuinely useful, less just "technically answering the question." This is the stage most responsible for the difference between a raw instruction-tuned model and something that feels like talking to a careful, thoughtful assistant.

So the full picture is:

```
Pretraining (predict next token on huge text corpus) → base model
↓
Fine-tuning (question → answer examples) → instruction-following model
↓
RLHF (human preference ranking) → helpful, aligned assistant
↓
[weights now FROZEN]
↓
Inference (your message → tokenize → loop through the Transformer,
           one token at a time, using frozen weights) → the answer you see
```

Quick check: given that the weights are frozen during inference, why do you think Claude can still hold a coherent multi-turn conversation with you — where is "memory" of earlier messages actually coming from, if the model itself isn't changing or storing anything between messages? (Hint: think back to what gets fed into step 1 of the loop above.)

